

Automated C Vulnerability Detection via Structure-Enhanced Graph Transformer and RBM

Yibo Zhou
ybzhou@nuist.edu.cn
Nanjing University of Information
Science and Technology
Nanjing, Jiangsu, China

Xiaolong Xu*
xlxu@nuist.edu.cn
Nanjing University of Information
Science and Technology
Nanjing, Jiangsu, China

Haolong Xiang
hlxiang@nuist.edu.cn
Nanjing University of Information
Science and Technology
Nanjing, Jiangsu, China

Xuyun Zhang
xuyun.zhang@mq.edu.au
Macquarie University
Sydney, New South Wales, Australia

Siyu Wu
siyuwu@nuist.edu.cn
Nanjing University of Information
Science and Technology
Nanjing, Jiangsu, China

Abstract

Code vulnerability detection aims to identify potential risks in code to prevent malicious attacks. Previous sequence-based models mostly focus on capturing local syntactic patterns, but they fail to model the non-linear structural dependencies and intricate context found in real-world code. Recent graph-based methods integrate structural information such as abstract syntax trees (AST) and control flow graphs (CFG). However, they often rely on shallow features of code and struggle to capture rich contextual semantic relations, which limits the effectiveness of detection. To address the issues above, we propose a novel framework, named Vul-GRT. We firstly design an unsupervised-based joint feature extraction method, which integrates Graph Attention Networks (GAT) and Restricted Boltzmann Machines (RBM) with joint training to capture structural information and further mine latent features from code graphs. Then, we leverage code language models to incorporate node-level semantic information, which is fused with structural features to obtain comprehensive node representations. Moreover, we introduce a hybrid loss function that integrates both vulnerability detection and path tracking objectives, which enables the model to focus on path context during vulnerability localization, enhancing the traceability of identified vulnerabilities. In the training step, we utilize Graph Transformer to model long-range dependencies and further capture critical relationships across the entire code graph. Finally, we conduct extensive experiments on two benchmark datasets, NVD and SARD, and experimental results demonstrate the effectiveness and robustness of our proposed framework compared with existing state-of-the-art methods.

*Corresponding author.

CCS Concepts

• **Computing methodologies** → **Learning latent representations; Multi-task learning; Software and its engineering** → **Software notations and tools.**

Keywords

Code Vulnerability Detection, Feature Extraction and Fusion, Hybrid Loss Function, Graph Transformer

ACM Reference Format:

Yibo Zhou, Xiaolong Xu, Haolong Xiang, Xuyun Zhang, and Siyu Wu. 2026. Automated C Vulnerability Detection via Structure-Enhanced Graph Transformer and RBM. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining (WSDM '26)*, February 22–26, 2026, Boise, ID, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3773966.3778005>

1 Introduction

With the rapid development of software systems, code vulnerabilities have become critical threats to the security and reliability of modern computing environments. Security flaws in code are often exploited by attackers for unauthorized access, data theft, privilege escalation, and large-scale cyberattacks, leading to significant financial and reputational damage to organizations [18]. Nowadays, the accurate and efficient identification of potential software vulnerabilities remains a central research challenge and a practical necessity for ensuring the reliability and security of modern software applications [11, 22, 30].

Traditional vulnerability detection approaches have typically centered around static and dynamic analysis, involving extensive manual rule-based inspections, pattern matching, or heuristic-driven checks [1, 3]. However, such techniques demand considerable domain expertise and are struggle to adapt the growing of codebases and the diversity of modern vulnerability patterns.

With the rise of deep learning, substantial progress has been made toward automated vulnerability analysis. Sequence-based models, have been explored to encode source code as token sequences [7, 21, 24, 25, 27]. These models target at learning local syntactic patterns and sequential dependencies, allowing for the automated detection of certain bug types and anti-patterns. However,



This work is licensed under a Creative Commons Attribution 4.0 International License. WSDM '26, Boise, ID, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2292-9/2026/02

<https://doi.org/10.1145/3773966.3778005>

such sequence-based models suffer from capturing the complex control flows and program dependencies in real software systems. Consequently, such sequence-based models often overlook the key vulnerability propagation paths across distant or context-dependent code regions.

Recent research has focused on graph-based learning paradigms, which provide a more expressive and structured representation of code [14, 23, 26, 29, 31]. By explicitly modeling high-level program artifacts, such as abstract syntax trees (ASTs), control flow graphs (CFGs), and data flow graphs (DFGs), these graph-based approaches focus on more syntactic structural information beyond simple sequence patterns. However, most existing studies still depend on manual feature engineering and regard shallow attributes as graph node representations, which hinder capturing the contextual semantics and limit the robustness and generalizability of these models.

To address the challenges above, we propose Vul-GRT, a novel end-to-end framework for code vulnerability detection. Firstly, we combine Graph Attention Networks (GAT) [19] and Restricted Boltzmann Machines (RBM) [16] for joint unsupervised feature extraction, allowing simultaneous modeling of both local and global relationships in code graphs. We further enhance node features by integrating semantic embeddings from pretrained code language models [10], adaptively fusing them with structure-based representations to create comprehensive, context-aware embeddings. Furthermore, we incorporate a Graph Transformer and a hybrid loss function to jointly optimize vulnerability detection and fine-grained trace attribution, which improves detection accuracy while considering interpretable vulnerability localization. Experiments on two benchmark datasets, NVD and SARD, show that Vul-GRT achieves competitive results compared to state-of-the-art methods, demonstrating its effectiveness and robustness in complex code vulnerability detection tasks.

The main contributions of this paper are summarized as follows:

- We propose a unified framework, Vul-GRT, that combines graph-based structural modeling and code semantics for robust code vulnerability detection.
- We introduce an unsupervised joint feature extraction method that integrates GAT and RBM, enabling the capture of both explicit graph structure and latent code features.
- We design a hybrid loss function that considers simultaneous program-level vulnerability detection and fine-grained trace localization, which enhances both accuracy and interpretability of detection.
- We conduct extensive experiments on widely-used vulnerability detection benchmarks (NVD and SARD), demonstrating the superior effectiveness and robustness of Vul-GRT compared to state-of-the-art baselines.

2 Related Work

2.1 Sequence-Based Methods

Early research on automated code vulnerability detection and code analysis predominantly utilized sequence-based deep learning models. These approaches treat source code as a sequence of tokens or characters and employ neural network architectures. Recurrent

Neural Networks (RNNs) are foundational models capable of capturing sequential dependencies by passing hidden states through the sequence [2]. Long Short-Term Memory (LSTM) networks incorporate memory cells and gating mechanisms to better capture long-term dependencies and complex syntactic relationships within code snippets [5, 35]. Subsequently, Gated Recurrent Units (GRU) introduced a streamlined gating mechanism that improves computational efficiency while still effectively modeling sequential information and mitigating vanishing gradient problems [12]. In addition to these, Convolutional Neural Networks (CNNs) have also been explored for code classification and vulnerability detection by focusing on local features [17]. While these methods have demonstrated notable success in capturing shallow syntactic and lexical relationships, they often struggle to effectively model complex, non-linear structural dependencies inherent in program logic, which restricts their capacity to capture long-range interactions and potentially missing critical vulnerability propagation paths.

2.2 Graph-Based Methods

To address the limitations of sequence-based models, researchers have increasingly turned to graph-based approaches, which explicitly capture the rich, structured nature of source code. By representing programs as various types of graphs, such as Abstract Syntax Trees (AST), Control Flow Graphs (CFG), or Data Flow Graphs (DFG), these methods allow for the preservation of complex control and data dependencies beyond simple token sequences [4]. DR-GCN [34] aggregates information from neighboring nodes through hierarchical convolutions, enabling the model to learn meaningful structural patterns in codes. Devign [33] introduces attention mechanisms that assign varying importance to neighboring nodes, providing more context-aware information aggregation. TS-GGNN [32] employs gating mechanisms to better capture iterative and sequential flows within code graphs. ASTNN [28] improves hierarchical structure modeling by dividing large abstract syntax trees into statement-level subtrees and encoding them with neural networks. However, many existing graph-based approaches still rely on shallow, hand-crafted node features or static code metrics, which lacks the capacity to fully capture deep semantic interactions and the rich contextual information present between code components, limiting their effectiveness in complex vulnerability detection tasks.

2.3 Code Language Models

In parallel with advancements in program analysis, pretrained code language models have recently emerged as powerful tools for representing source code semantics. CodeBERT [6] leverages paired code and natural language data using masked language modeling and replaced token detection to learn contextual representations. GraphCodeBERT [9] integrates data flow graph information and focus on model structural relationships, improving the performance in code comprehension and vulnerability detection. CodeT5 [20] adopts an encoder-decoder framework and formulates various code tasks as text-to-text problems, enabling effective multitask learning across code translation, summarization, and defect prediction. While these models have achieved state-of-the-art results on many benchmarks, they primarily capture sequence-level semantics and

often require additional structural encoders or hybrid approaches to fully utilize the rich structure inherent in source code.

3 Problem Definition

Given a C source program, we first parse and represent it as a directed, attributed code graph $G = (V, E, \mathbf{X})$, where V is the set of nodes corresponding to code entities (e.g., statements, functions, or variables), E encodes structural relations such as control flow and data dependencies, and $\mathbf{X} \in \mathbb{R}^{|V| \times d}$ denotes the node feature matrix aggregating lexical, syntactic, and semantic information. This graph-based representation captures both local and long-range interactions for vulnerability analysis. Our framework addresses two supervised tasks: (1) graph-level vulnerability detection, predicting whether the entire input code contains any vulnerabilities, formalized by a binary label $y_G \in \{0, 1\}$ assigned to each graph, and (2) node-level vulnerability localization (trace), where nodes v_i are annotated as vulnerable or not via labels $y_i \in \{0, 1\}$. For a batch of graphs, we compute a predicted vulnerability probability $\hat{y}_G$ at the graph level and node-wise logits from which per-node probabilities are derived. Model optimization is driven by a multi-task objective: binary cross-entropy loss for the graph-level prediction, and cross-entropy loss over all labeled nodes for localization, which can be formulated as follows:

$$\mathcal{L}_{\text{detect}} = -\frac{1}{B} \sum_b [y_G^b \log \hat{y}_G^b + (1 - y_G^b) \log(1 - \hat{y}_G^b)], \quad (1)$$

$$\mathcal{L}_{\text{trace}} = -\frac{1}{|M|} \sum_{(b,i) \in M} [y_i^b \log p_{i,1}^b + (1 - y_i^b) \log p_{i,0}^b], \quad (2)$$

where M is the set of valid annotated nodes and $p_{i,1}^b, p_{i,0}^b$ denote the predicted probability of node i in graph b being vulnerable or not. The final joint loss is represented as:

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{detect}} + \lambda \mathcal{L}_{\text{trace}}, \quad (3)$$

where λ is a parameter to balance the two objectives. Finally, we aim to minimize $\mathcal{L}_{\text{joint}}$, allowing the model to simultaneously detect whether a program is vulnerable and to precisely localize the vulnerable code segments.

4 Method

In this section, we present Vul-GRT, our unified framework vulnerability detection in C code, which integrates deep semantic understanding with graph-based representation learning. The overview of Vul-GRT is shown in Figure 1. Our framework consists of three main stages. Firstly, we develop an unsupervised joint feature extraction method that combines Graph Attention Networks (GAT) and Restricted Boltzmann Machines (RBM) with joint training, effectively capturing both local node interactions and global structural dependencies within the code graph. Secondly, we incorporate code language modeling by extracting semantic embeddings of the abstracted code statements for each node using a pretrained code language model. Then we fuse these semantic embeddings with the structure-aware node features, producing comprehensive and context-rich node representations. Thirdly, we further introduce a hybrid loss function that unifies vulnerability detection and trace attribution objectives, enabling the model to jointly optimize for both

program-level risk assessment and precise path-level localization. The fused node representations are fed into a Graph Transformer, which is optimized by the hybrid loss function, to model long-range dependencies across the entire code graph. For additional algorithmic details, we refer readers to Appendix A. In the following subsections, we will describe each stage of our framework in detail and provide the corresponding technical formulations.

4.1 Unsupervised-Based Joint Feature Extraction via GAT and RBM

4.1.1 GAT-based Node Representation. The initial stage of our framework aims to construct powerful, type-sensitive node representations for the code graph by combining local and global structural modeling. To this end, we leverage a Graph Attention Network (GAT) to encode node features, and further utilize a Restricted Boltzmann Machine (RBM) to regularize the global structure. Crucially, instead of direct node-type classification, we supervise the learned representations using positive and negative edge sampling, thus explicitly modeling the likelihood of edge existence in the code graph.

To capture complex relationships among code entities, we apply a L -layer multi-head GAT to the node feature matrix $\mathbf{X}$. The GAT layer enables each node to dynamically attend to its neighbors via learnable attention weights, thus modeling non-uniform interaction strength among code elements. For a GAT layer l and attention head k :

$$\mathbf{h}_i^{(l+1)} = \parallel \left\| \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(l,k)} \mathbf{W}^{(l,k)} \mathbf{h}_j^{(l)} \right\|, \quad (4)$$

where $\parallel$ denotes vector concatenation, $\sigma(\cdot)$ is an activation function, $\mathcal{N}(i)$ denotes the neighborhood of node i , $\mathbf{W}^{(l,k)} \in \mathbb{R}^{d \times d}$ are head-specific linear transformations. Then, $\alpha_{ij}^{(l,k)}$ is computed as:

$$\alpha_{ij}^{(l,k)} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^{(l,k)\top} [\mathbf{W}^{(l,k)} \mathbf{h}_i^{(l)} \parallel \mathbf{W}^{(l,k)} \mathbf{h}_j^{(l)}]))}{\sum_{j' \in \mathcal{N}(i)} \exp(\cdot)}, \quad (5)$$

where $\cdot$ denotes the numerator expression inside the exponential function. By stacking GAT layers, each node in the graph aggregates local and contextual information progressively.

4.1.2 Latent Feature Representation Mining via RBM. To explicitly incorporate more latent features, we employ RBM over the set of node representations after GAT layer. Firstly, for every training step, we sample a set of positive edges E_{pos} from the true graph edges E (i.e., $(v_i, v_j) \in E$), and then sample an equal-size set of negative edges E_{neg} by randomly pairing nodes that do not have an edge (i.e., $(v_i, v_j) \notin E$). Each edge e_{ij} is represented as:

$$\mathbf{e}_{ij} = [\mathbf{h}_i \parallel \mathbf{h}_j]. \quad (6)$$

For edge e_{ij} , we define label $y_{ij} = 1$ if positive, and 0 if negative. Then we employ a sigmoid classifier to predict edge probability, which is formulated as:

$$\hat{y}_{ij} = \sigma(\mathbf{w}_e^\top \mathbf{e}_{ij} + b_e), \quad (7)$$

$$\mathcal{L}_{\text{edge}} = -\frac{1}{|E_s|} \sum_{e_{ij} \in E_s} [y_{ij} \log \hat{y}_{ij} + (1 - y_{ij}) \log(1 - \hat{y}_{ij})], \quad (8)$$

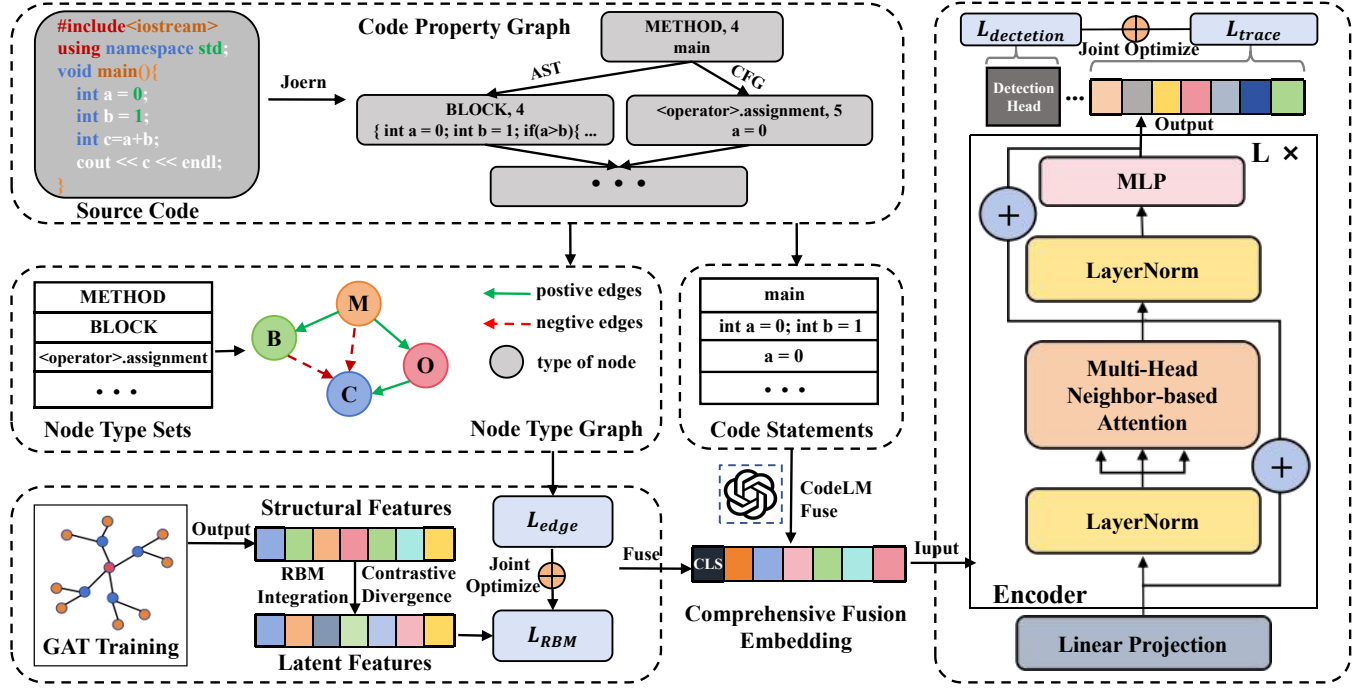


Figure 1: Overview of our proposed Vul-GRT framework, which contains four stages: (1) Firstly, we process the source code into code property graph; (2) Secondly, we conduct unsupervised-based joint extraction via GAT and RBM to further mine latent structural features; (3) Thirdly, we obtain the comprehensive embedding by fusing the previous structural features with contextual semantic embedding from code language models. (4) Finally, we perform graph-level vulnerability classification and node-level trace localization via Graph Transformer through hybrid loss function.

where $E_s = E_{pos} \cup E_{neg}$. The edge classification loss is computed as the average binary cross-entropy over both positive and negative sampled edges. For $\mathbf{H} = [\mathbf{h}_1, \dots, \mathbf{h}_{|V|}]$:

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{v}^\top \mathbf{W}_{\text{RBM}} \mathbf{h} - \mathbf{b}_v^\top \mathbf{v} - \mathbf{b}_h^\top \mathbf{h}, \quad (9)$$

where $\mathbf{v}$ stacks node vectors as visible units. The RBM is trained via contrastive divergence, with its loss denoted as $\mathcal{L}_{\text{RBM}}$. The total multi-task loss is represented as:

$$\mathcal{L}_{\text{rep}} = \alpha \mathcal{L}_{\text{edge}} + \beta \mathcal{L}_{\text{RBM}}, \quad (10)$$

where α and β are hyperparameters that balance the two loss terms. Through our proposed unsupervised joint feature extraction method, we enable comprehensive modeling of both explicit structural dependencies and latent high-level patterns within code graphs, which enhances the robustness of the representations across diverse code scenarios.

4.2 Semantic Embedding Fusion with Code Language Models

To further enhance the expressive capacity of graph-based node representations, we integrate semantic information derived from a large pretrained code language model. Specifically, we employ a code language model (CodeLM) to encode the semantics of each code statement s_i associated with nodes in the graph, and fuse

these embeddings with the structure-aware features obtained in the previous stage. Such process is represented as:

$$\mathbf{e}_i = \text{CodeLM}(s_i), \quad (11)$$

$$\mathbf{f}_i = [\mathbf{z}_i \| \mathbf{e}_i], \quad (12)$$

where $\mathbf{z}_i$ denotes the node feature vector of v_i learned from the previous stage, and $\mathbf{e}_i$ denotes the statement-level embedding output by the encoder of the code language model. By fusing structure-enhanced node features with abstract code semantic embeddings, we obtain the final representation for each node $\mathbf{f}_i$, which is then used as the comprehensive embedding for downstream vulnerability detection and localization.

4.3 Joint Vulnerability Detection and Localization with Graph Transformer

In this stage, we utilize the fused node representations as input to a Graph Transformer, which effectively captures long-range relationships within the code graph. Moreover, we propose a hybrid loss function that jointly optimizes both vulnerability detection and path tracing objectives, which further enhances the traceability of identified vulnerabilities.

Specifically, let $\mathbf{F} = [\mathbf{f}_1; \mathbf{f}_2; \dots; \mathbf{f}_{|V|}]$ denote the matrix of node embeddings from the previous fusion step. We also introduce a learnable token $\mathbf{f}_{[\text{CLS}]}$ that is concatenated to this matrix, which is

represented as:

$$\mathbf{F}^{(0)} = [\mathbf{f}_{[\text{CLS}]}; \mathbf{f}_1; \dots; \mathbf{f}_{|V|}]. \quad (13)$$

The [CLS] token serves as an aggregator of global contextual information across the entire graph, enabling effective graph-level prediction. Then, we employ Graph Transformer for further modeling, each layer performs global node interaction and feature propagation via multi-head self-attention and feed-forward layers, coupled with residual connections and layer normalization. For layer l , head m , and each node i , the update is formulated as:

$$\mathbf{Q}_i^{(m)} = \mathbf{f}_i^{(l)} \mathbf{W}_Q^{(m)}, \mathbf{K}_j^{(m)} = \mathbf{f}_j^{(l)} \mathbf{W}_K^{(m)}, \mathbf{V}_j^{(m)} = \mathbf{f}_j^{(l)} \mathbf{W}_V^{(m)}, \quad (14)$$

$$\alpha_{ij}^{(m)} = \frac{\exp\left(\frac{\mathbf{Q}_i^{(m)} \cdot (\mathbf{K}_j^{(m)})^\top}{\sqrt{d_k}}\right)}{\sum_{j'} \exp\left(\frac{\mathbf{Q}_i^{(m)} \cdot (\mathbf{K}_{j'}^{(m)})^\top}{\sqrt{d_k}}\right)}, \quad (15)$$

$$\text{Attention}(\mathbf{F}^{(l)}) = \text{Concat}_{m=1}^M \left(\sum_j \alpha_{ij}^{(m)} \mathbf{V}_j^{(m)} \right), \quad (16)$$

where M denotes the number of attention heads, and $\mathbf{W}_Q^{(m)}$, $\mathbf{W}_K^{(m)}$, and $\mathbf{W}_V^{(m)}$ are learnable weights for the m -th head. $\alpha_{ij}^{(m)}$ is calculated by the softmax operation, which allows the model to focus on the most relevant interactions of the code sequence. After concatenating the outputs of all M attention heads, the final outputs will pass through residual connection layers and Layer Normalization(LayerNorm) layers, which is represented as:

$$\mathbf{F}^{(l,*)} = \text{LayerNorm}\left(\mathbf{F}^{(l)} + \text{Attention}(\mathbf{F}^{(l)})\right), \quad (17)$$

$$\mathbf{F}^{(l+1)} = \text{LayerNorm}\left(\mathbf{F}^{(l,*)} + \text{FFN}(\mathbf{F}^{(l,*)})\right). \quad (18)$$

After L layers, we obtain the final node representations: $\mathbf{F}^{(L)} = [\mathbf{f}_{[\text{CLS}]}^{(L)}; \mathbf{f}_1^{(L)}; \dots; \mathbf{f}_{|V|}^{(L)}]$. The final [CLS] token embedding is used for whole-graph binary classification. Meanwhile, each node embedding is used for node-wise binary classification. Such process is formulated as:

$$\hat{y}_{\text{graph}} = \sigma\left(\mathbf{w}_g^\top \mathbf{f}_{[\text{CLS}]}^{(L)} + b_g\right), \quad (19)$$

$$\hat{y}_i = \sigma\left(\mathbf{w}_n^\top \mathbf{f}_i^{(L)} + b_n\right), \quad \forall i \in V, \quad (20)$$

where $\sigma(\cdot)$ denotes the sigmoid function, $\mathbf{w}_g$, $\mathbf{w}_n$, b_g , and b_n are learnable parameters. Our model is optimized using a hybrid loss that integrates both vulnerability detection and path tracking objectives, which is calculated as:

$$\mathcal{L}_{\text{detect}} = -[y_{\text{graph}} \log \hat{y}_{\text{graph}} + (1 - y_{\text{graph}}) \log(1 - \hat{y}_{\text{graph}})], \quad (21)$$

$$\mathcal{L}_{\text{trace}} = -\frac{1}{|V'|} \sum_{i \in V'} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)], \quad (22)$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{detect}} + \lambda \mathcal{L}_{\text{trace}}, \quad (23)$$

where $\lambda > 0$ balances the two objectives, and V' is the subset of nodes with ground truth trace annotation.

In summary, our proposed Vul-GRT integrates structural and semantic code features through deep fusion, enables comprehensive global reasoning over program graphs. By leveraging the [CLS] token for holistic vulnerability detection and node-level outputs for fine-grained localization, Vul-GRT achieves both robust detection performance and precise identification of vulnerable code traces.

5 Experiments

5.1 Experimental Setup

5.1.1 Datasets. We use two real-world commonly used benchmark datasets for model evaluation. The NVD (National Vulnerability Database) dataset consists of real-world vulnerable code samples collected from open-source projects and documented CVEs, with each function-level snippet labeled as either vulnerable or not. The SARD (Software Assurance Reference Dataset) includes both real and synthetic code samples, where vulnerabilities are either manually injected or sourced from real cases.

5.2 Implementation Details.

The technical modules of Vul-GRT primarily consist of joint feature extraction via GAT and RBM, semantic embedding fusion with code language models, and hybrid loss optimized training via Graph Transformer. We refer the reader to Appendix B for more details of the experimental parameter settings.

5.2.1 Metrics. In this paper, we evaluate our models using four standard metrics: accuracy, precision, recall, and F1-score. Specifically, accuracy measures the overall proportion of correctly classified samples; precision indicates how many samples predicted as vulnerable are actually vulnerable; recall represents how many real vulnerabilities are correctly identified; F1-score provides a balanced measure between precision and recall. All the metrics we used are calculated as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \quad (24)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (25)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (26)$$

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (27)$$

where TP (True Positive) refers to vulnerable samples correctly identified as vulnerable; TN (True Negative) refers to non-vulnerable samples correctly recognized as non-vulnerable; FP (False Positive) means non-vulnerable samples incorrectly predicted as vulnerable; FN (False Negative) means vulnerable samples that are incorrectly predicted as non-vulnerable.

5.2.2 Baselines. To evaluate the effectiveness of our proposed method, we compare Vul-GRT with several baselines. In this paper, all baselines are trained and evaluated using the same data split and metrics for a fair comparison, which are listed as follows:

Table 1: Performance comparison (Accuracy, Precision, Recall, F1-Score in %) of different vulnerability detection methods on NVD and SARD datasets. The best results are bold and the second best are underlined.

Methods \ Datasets	NVD				SARD			
	Accuracy(%)	Precision(%)	Recall(%)	F1-Score(%)	ACC(%)	Precision(%)	Recall(%)	F1-Score(%)
VulDeePecker (2018)	78.43	76.20	71.85	73.96	74.25	73.32	68.11	70.62
SySeVR (2022)	83.15	81.62	73.44	77.31	80.42	81.95	69.68	74.56
Devign (2019)	81.88	79.51	75.30	77.35	80.94	79.25	67.56	72.94
μ VulDeePecker (2021)	81.21	80.48	73.78	76.98	78.72	80.61	70.95	75.47
CodeBERT (2020)	85.11	86.94	75.40	80.76	82.37	81.53	72.71	76.87
UniXcoder (2022)	86.74	84.18	<u>78.45</u>	81.21	82.55	81.02	74.10	77.41
CasualVul (2024)	<u>87.85</u>	84.53	78.22	<u>81.25</u>	<u>84.03</u>	<u>82.62</u>	<u>75.15</u>	<u>78.71</u>
Vul-GRT (Ours)	88.51	<u>86.44</u>	80.08	83.14	86.15	83.30	79.95	81.59

- VulDeePecker: VulDeePecker [13] learns features from code gadgets semantically related code fragments without manual feature engineering. It effectively reduces missed vulnerabilities and can uncover real vulnerabilities that other approaches often overlook.
- SySeVR: SySeVR [12] focuses on vector representations and represents C/C++ code by incorporating both syntax and semantic information. It demonstrates strong effectiveness by uncovering previously unknown vulnerabilities missed by existing approaches.
- Devign: Devign [33] is a GNN-based model that identifies vulnerable functions in source code by learning from rich code semantic representations, which achieves notably higher accuracy.
- μ VulDeePecker: μ VulDeePecker [35] is designed for multi-class vulnerability detection. By leveraging code attention mechanisms, it achieves effective fine-grained vulnerability classification beyond traditional binary detection methods.
- CodeBERT: CodeBERT [6] is a bimodal pre-trained model based on Transformers that learns joint representations of programming and natural language, enabling powerful downstream tasks like code search and documentation generation.
- UniXcoder: UniXcoder [8] is a unified cross-modal pre-trained model for programming languages that leverages both code, comments, and AST information to support a range of code understanding and generation tasks.
- CasualVul: CasualVul [15] introduces causal learning for vulnerability detection, enabling models to avoid relying on non-robust features. This approach significantly improves model’s robustness and generalization compared to existing methods.

5.3 Result Analysis

The experimental results presented in Table 1 demonstrate that our proposed Vul-GRT consistently outperforms existing state-of-the-art vulnerability detection methods across both the NVD and SARD

datasets. Compared with traditional approaches such as VulDeePecker and SySeVR, Vul-GRT achieves substantial improvements in all evaluation metrics, indicating a significant advance in both detection accuracy and robustness.

Comparing with recent strong baselines, such as CodeBERT and UniXcoder, which leverage pretrained code language models, Vul-GRT attains notable gains, particularly in terms of recall and F1-score. This suggests that the incorporation of semantic embedding fusion combined with graph transformer architectures enables the model to better capture complex code semantics and structural dependencies, thus identifying more true vulnerabilities while maintaining high precision.

Moreover, Vul-GRT improves upon CasualVul, a competitive causal learning-based model, reflecting the effectiveness of our joint detection and localization framework coupled with the hybrid loss optimization. These enhancements translate into more balanced performance, reducing false negatives without substantially increasing false positives, which is essential for practical vulnerability analysis.

The consistent superiority of Vul-GRT across two diverse datasets further substantiates its generalization capability and robustness to different programming environments and vulnerability patterns, making it a promising solution for real-world security analysis tasks.

5.4 Ablation Study Analysis

To evaluate the contributions of each core component in the Vul-GRT framework, we conducted a series of ablation experiments, summarized in Table 2. The complete Vul-GRT model, which incorporates RBM module, semantic embedding fusion, and joint loss optimization, achieves superior performance across all evaluated metrics.

5.4.1 Impact of Semantic Embedding Fusion(SEF). Removing the semantic embedding fusion component results in a statistically significant decline across all performance metrics. Specifically, the accuracy drops by 3.22%, while precision decreases by 2.33%, and

Table 2: Ablation study on key components of Vul-GRT showing the impact on Accuracy (ACC), Precision (P), Recall (R), and F1-Score (F1) on the NVD dataset.

Model Variants	ACC (%)	P (%)	R (%)	F1 (%)
Full Vul-GRT (w/ RBM)	88.51	86.44	80.08	83.14
Full Vul-GRT (w/o RBM)	87.34	85.98	79.56	82.65
w/o SEF (w/ RBM)	85.29	84.11	75.63	79.64
w/o SEF (w/o RBM)	84.56	83.59	74.21	78.62
w/o Trace Loss (w/ RBM)	84.68	83.05	73.10	77.76
w/o Trace Loss (w/o RBM)	83.74	81.29	73.17	77.02

recall experiences an even more pronounced reduction. Such decline demonstrates the critical importance of semantic information derived from pretrained code language models in enhancing feature expressiveness, which enables the model to detect subtle vulnerabilities that traditional structural features may overlook.

5.4.2 Effect of RBM Module Integration. The variant of the model that incorporates the RBM exhibits a significant improvement in accuracy and F1 score compared to its counterpart that lacks RBM integration. The RBM’s ability to mine latent representations of input data by capturing complex relationships among features significantly enhances the model’s detection efficiency. This integration of RBM not only improves performance metrics but also facilitates better feature learning, which is vital for identifying nuanced vulnerabilities.

5.4.3 Trace Loss Contribution. Excluding the trace loss component from the hybrid optimization framework results in marked reductions in both recall and F1 score. This outcome underscores the importance of joint learning for vulnerability detection and path tracing, which serves as an effective regularizer. By emphasizing both graph-level classifications and fine-grained, node-level vulnerability localization, Vul-GRT achieves a more comprehensive understanding of potential vulnerabilities within code structures.

In conclusion, each of these components plays a crucial role in enhancing the overall effectiveness of Vul-GRT, underscoring the importance of their integration to achieve optimal performance in vulnerability detection tasks.

5.5 Sensitivity Analysis

We further present a comprehensive sensitivity analysis of the hybrid loss balancing parameter λ utilized in our Vul-GRT model. The experiments involve evaluating a range of λ values from 1×10^{-3} to 1.0, and the resulting performance metrics are systematically reported in Figure 2.

The assessment begins with a low value of λ , specifically 1×10^{-3} . Vul-GRT achieves an accuracy of 87.63%, along with precision and recall values of 85.67% and 79.16%. While these metrics are reasonably acceptable, the recall indicates a potential inadequacy in detecting sensitive nodes. As λ increases to 5×10^{-3} yields notable improvements in performance, with accuracy rising to 88.35% and

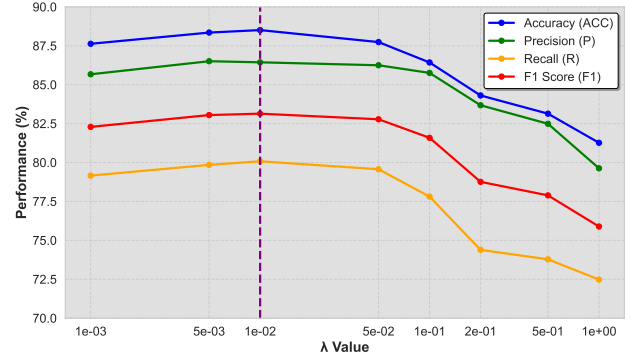


Figure 2: Sensitivity analysis of λ in the hybrid loss function. Vul-GRT achieves optimal performance at $\lambda = 1 \times 10^{-2}$, balancing global vulnerability detection and local trace identification.

recall increasing to 79.85%, which demonstrates Vul-GRT begins to leverage the benefits of trace loss effectively. At $\lambda = 1 \times 10^{-2}$, the model achieves peak performance, with accuracy, precision, recall, and F1 score values of 88.51%, 86.44%, 80.08%, and 83.14%. As we further increase λ to 5×10^{-2} , the model’s performance remains acceptable, with an accuracy of 87.74%. However, there is a slight decline in precision (86.25%) and recall (79.57%), suggesting a potential shift away from both node-level details and global detection capabilities. Increasing λ to higher values results in a significant performance decline, reflecting an overemphasis on node-level details that degrades overall detection capabilities.

6 Conclusion

In this paper, we propose Vul-GRT, a novel and robust framework designed to significantly enhance the performance of vulnerability detection in C code. To achieve a comprehensive understanding of intricate code structure patterns, we propose a sophisticated joint feature extraction method. This approach is further refined through the integration of semantic embedding fusion, which demonstrably enriches the contextual awareness of potential vulnerabilities. Moreover, we introduce a carefully formulated hybrid loss function, engineered to simultaneously achieve highly accurate detection of security flaws. Extensive experimental results unequivocally showcase Vul-GRT’s superior performance across various benchmarks. Furthermore, detailed ablation studies confirm the critical and synergistic contributions of each technical component to the overall efficacy of our framework. In the future, we will focus on extending Vul-GRT’s capabilities to multiple programming languages and developing intuitive visualization tools to facilitate a more profound analysis and intuitive presentation of detected vulnerabilities.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62372242, and Jiangsu Provincial Major Project on Basic Research of Cutting-edge and Leading Technologies, under grant no. BK20232032.

References

- [1] Nathaniel Ayewah, William Pugh, David Hovemeyer, J. David Morgenthaler, and John Penix. 2008. Using Static Analysis to Find Bugs. *IEEE Software* 25, 5 (2008), 22–29.
- [2] Mohammad Taneem Bin Nazim, Md Jobair Hossain Faruk, Hossain Shahriar, Md Abdullah Khan, Mohammad Masum, Nazmus Sakib, and Fan Wu. 2022. Systematic Analysis of Deep Learning Model for Vulnerable Code Detection. In *2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*. 1768–1773.
- [3] B. Chess and G. McGraw. 2004. Static analysis for security. *IEEE Security & Privacy* 2, 6 (2004), 76–79.
- [4] Lei Cui, Zhiyu Hao, Yang Jiao, Haiqiang Fei, and Xiaochun Yun. 2021. VulDetector: Detecting Vulnerabilities Using Weighted Feature Graph Comparison. *IEEE Transactions on Information Forensics and Security* 16 (2021), 2004–2017.
- [5] Hoa Khanh Dam, Truyen Tran, Trang Pham, Shien Wee Ng, John Grundy, and Aditya Ghose. 2021. Automatic Feature Learning for Predicting Vulnerable Software Components. *IEEE Transactions on Software Engineering* 47, 1 (2021), 67–85.
- [6] Zhangjun Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547.
- [7] Yasir Glani, Luo Ping, Syed Asad Shah, and Lin Ke. 2025. CCDive: A Deep Dive into Code Clone Detection Using Local Sequence Alignment. *Tsinghua Science and Technology* 30, 4 (2025), 1435–1456.
- [8] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7212–7225.
- [9] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*.
- [10] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and Evaluating Contextual Embedding of Source Code. In *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. 5110–5121.
- [11] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre Attacks: Exploiting Speculative Execution. In *2019 IEEE Symposium on Security and Privacy (SP)*. 1–19.
- [12] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2244–2258.
- [13] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18–21, 2018*.
- [14] Xiangmao Meng, Xinqiang Wen, Xinliang Sun, Ju Xiang, Yahui Long, Xuan Lin, and Min Li. 2025. SAGAN: A Subgraph-Aware Graph Attention Network for Drug repositioning. *Tsinghua Science and Technology* (2025).
- [15] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. 2024. Towards Causal Deep Learning for Vulnerability Detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Article 153, 11 pages.
- [16] Ruslan Salakhutdinov, Andriy Mnih, and Geoffrey Hinton. 2007. Restricted Boltzmann machines for collaborative filtering (*ICML '07*). 791–798.
- [17] Christoforos Seas, Glenn Fitzpatrick, John A. Hamilton, and Martin C. Carlisle. 2024. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*.
- [18] Mario Calin Sánchez, Juan Manuel Carrillo de Gea, José Luis Fernández-Alemán, Jesús Garcerán, and Ambrosio Toval. 2020. Software vulnerabilities overview: A descriptive study. *Tsinghua Science and Technology* 25, 2 (2020), 270–280.
- [19] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph Attention Networks. *ArXiv abs/1710.10903* (2017).
- [20] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 8696–8708.
- [21] Martin White, Michele Tufano, Christopher Vendome, and Denys Poshyvanyk. 2016. Deep learning code fragments for code clone detection. In *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 87–98.
- [22] Haolong Xiang, Xuyun Zhang, Hongsheng Hu, Lianying Qi, Wanchun Dou, Mark Dras, Amin Beheshti, and Xiaolong Xu. 2023. OptiForest: Optimal Isolation Forest for Anomaly Detection. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*. 2379–2387.
- [23] Xiaolong Xu, Yuxin Cao, Hongsheng Hu, Haolong Xiang, Lianying Qi, Junqun Xiong, and Wanchun Dou. 2025. MGF-ESE: An Enhanced Semantic Extractor with Multi-Granularity Feature Fusion for Code Summarization. In *THE WEB CONFERENCE 2025*.
- [24] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Xiaodong Song. 2017. Neural Network-based Graph Embedding for Cross-Platform Binary Code Similarity Detection. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (2017)*.
- [25] Xiaolong Xu, Yibo Zhou, Haolong Xiang, Xiaoyong Li, Xuyun Zhang, Lianying Qi, and Wanchun Dou. 2025. NLGT: Neighborhood-based and Label-enhanced Graph Transformer Framework for Node Classification. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*. 12954–12962.
- [26] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *2014 IEEE Symposium on Security and Privacy*. 590–604.
- [27] Fabian Yamaguchi, Felix "FX" Lindner, and Konrad Rieck. 2011. Vulnerability Extrapolation: Assisted Discovery of Vulnerabilities Using Machine Learning. In *5th USENIX Workshop on Offensive Technologies (WOOT 11)*.
- [28] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. 2019. A Novel Neural Source Code Representation Based on Abstract Syntax Tree. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 783–794.
- [29] Kechi Zhang, Wenhan Wang, Huangzhao Zhang, Ge Li, and Zhi Jin. 2022. Learning to Represent Programs with Heterogeneous Graphs. In *2022 IEEE/ACM 30th International Conference on Program Comprehension (ICPC)*.
- [30] Yifan Zhang, Haolong Xiang, Xiaolong Xu, Zishun Rui, Xiaoyong Li, Lianying Qi, and Fei Dai. 2025. MEGAD: A Memory-Efficient Framework for Large-Scale Attributed Graph Anomaly Detection. In *International Joint Conference on Artificial Intelligence*.
- [31] Huanjing Zhao, Pinde Rui, Jie Chen, Shu Zhao, and Yanping Zhang. 2025. Restage: Relation Structure-Aware Hierarchical Heterogeneous Graph Embedding. *Tsinghua Science and Technology* 30, 1 (2025), 198–214.
- [32] Xin Zhou, Jianmin Pang, Chunyan Zhang, Feng Yue, Junchao Wang, and Guangming Liu. 2023. TS-GGNN: Combining Graph and Sequence Features for Vulnerability Detection in Source Code. In *2023 5th International Conference on Communications, Information System and Computer Engineering (CISCE)*. 196–200.
- [33] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada*. 10197–10207.
- [34] Yuan Zhuang, Zhenguang Liu, Peng Qian, Qi Liu, Xiang Wang, and Qinming He. 2020. Smart Contract Vulnerability Detection using Graph Neural Network. In *International Joint Conference on Artificial Intelligence*.
- [35] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2021. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2021), 2224–2236.

A Algorithms

We develop an unsupervised joint feature extraction method that combines Graph Attention Networks (GAT) and Restricted Boltzmann Machines (RBM) with joint training, effectively capturing both local node interactions and global structural dependencies within the code graph. The specific process of algorithm is shown as Algorithm 1.

We also incorporate code language modeling by extracting semantic embeddings of the abstracted code statements for each node using a pretrained code language model. Then we fuse these semantic embeddings with the structure-aware node features, producing comprehensive and context-rich node representations. We further introduce a hybrid loss function that unifies vulnerability detection and trace attribution objectives, enabling the model to jointly optimize for both program-level risk assessment and precise path-level localization. The fused node representations are fed into a Graph Transformer, which is optimized by the hybrid loss function, to

Algorithm 1 Global Structural Representation Learning via GAT and RBM

Require: Code graph $G = (V, E, X)$, GAT layers L , positive/negative edge sample size N .
Ensure: Final node representations $\{z_i\}_{i \in V}$.

```

1: for each node  $v_i \in V$  do
2:   Initialize node feature  $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ .
3: end for
4: Sample positive edges  $E^+ \subset E$  and negative edges  $E^- \subset V \times V \setminus E$ , each of size  $N$ .
5: for  $l = 1$  to  $L$  do
6:   for each node  $v_i \in V$  do
7:     Update  $\mathbf{h}_i^{(l)}$  via GAT attention over neighbors.
8:   end for
9:   Get current node embeddings:  $\mathbf{h}_i = \mathbf{h}_i^{(l)}$ .
10:  for each edge  $(v_i, v_j)$  in  $E^+ \cup E^-$  do
11:    Form edge vector  $\mathbf{e}_{ij} = [\mathbf{h}_i \parallel \mathbf{h}_j]$ ,
12:    Predict edge score  $\hat{y}_{ij} = \sigma(\mathbf{w}_e^\top \mathbf{e}_{ij} + b_e)$ .
13:  end for
14:  Compute edge loss  $\mathcal{L}_{\text{edge}}$  via binary cross-entropy.
15:  Use  $\{\mathbf{h}_i\}$  as visible units for RBM.
16:  Apply RBM to obtain enhanced node embeddings  $\mathbf{z}_i$  and compute RBM loss  $\mathcal{L}_{\text{RBM}}$ .
17:  Minimize  $\mathcal{L}_{\text{rep}} = \alpha \mathcal{L}_{\text{edge}} + \beta \mathcal{L}_{\text{RBM}}$  and update parameters.
18: end for
19: Return final node representations  $\{z_i\}$ 

```

model long-range dependencies across the entire code graph. The specific process of algorithm is shown as Algorithm 2.

Algorithm 2 Feature Fusion and Graph Transformer Joint Training

Require: Node representations $\{z_i\}$, abstracted statements $\{s_i\}$, graph (V, E) , pretrained code language model *CodeLM*, GT layers L' .

```

1: for each node  $v_i \in V$  do
2:   Obtain node-level statement semantic embedding  $\mathbf{e}_i = \text{CodeLM}(s_i)$ ,
3:   Concatenate:  $\mathbf{f}_i = [z_i \parallel \mathbf{e}_i]$ .
4: end for
5: Form input matrix  $\mathbf{F} = [\mathbf{f}_{\text{CLS}}, \mathbf{f}_1, \dots, \mathbf{f}_{|V|}]$ .
6: for  $l = 1$  to  $L'$  do
7:   for each node  $i$  do
8:     Update  $\mathbf{f}_i$  with GT layer.
9:   end for
10: end for
11: Obtain  $\mathbf{f}_{\text{CLS}}^{(L')}$  and  $\mathbf{f}_i^{(L')}$  for all  $i$ 
12: Predict graph label:  $\hat{y}_{\text{graph}} = \sigma(\mathbf{w}_g^\top \mathbf{f}_{\text{CLS}}^{(L')} + b_g)$ 
13: for each node  $i \in V$  do
14:   Predict node label:  $\hat{y}_i = \sigma(\mathbf{w}_n^\top \mathbf{f}_i^{(L')} + b_n)$ 
15: end for
16: Compute loss  $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{detect}} + \lambda \mathcal{L}_{\text{trace}}$ 
17: Minimize  $\mathcal{L}_{\text{total}}$  and update parameters.

```

B Implementation Details

The GAT component consists of 4 layers, each equipped with 4 attention heads to capture multi-scale structural patterns, producing node representations with a dimensionality of 64. These GAT outputs are then regularized through a 2-layer restricted Boltzmann machine (RBM) with 128 hidden and visible units, which helps mining latent feature representations in the code graph. For semantic feature extraction, we leverage CodeT5 to generate contextual embeddings with a fixed dimension of 256. These learned code representations are then fused with the structural features from GAT through a linear projection layer, creating comprehensive node embeddings that encode both syntactic and semantic information. Our model is trained using a multi-task learning objective that combines edge classification loss ($\mathcal{L}_{\text{edge}}$) and RBM reconstruction loss ($\mathcal{L}_{\text{RBM}}$), weighted by $\alpha = 0.5$ and $\beta = 1 \times 10^{-2}$ respectively. Through extensive sensitivity analysis, we set the hybrid loss balancing parameter λ to 1×10^{-2} , which optimally trades off between global vulnerability detection and local trace identification tasks.

All experiments are conducted on an NVIDIA L40s GPU, using the AdamW optimizer with a learning rate of 5×10^{-4} and weight decay of 1×10^{-2} . We train the model for a maximum of 50 epochs with a batch size of 32, employing early stopping with a patience of 5 epochs to prevent overfitting. For regularization, we apply dropout ($p=0.1$) and layer normalization across all network layers.